Optimization

3.2.4 Box's method — a simplified version of EVOP (evolutionary operation)

José Firmino Aguilar Madeira

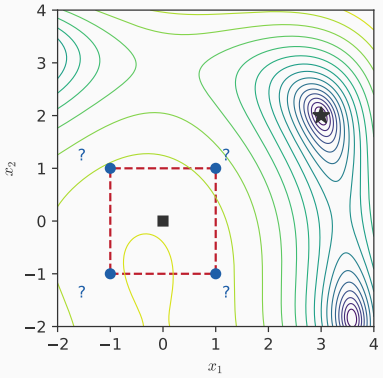
From a shape-changing simplex to a box that does not change

Nelder–Mead adapts the shape of the simplex using the values of f. What if we always keep the same geometric pattern?

In 2D, around the current point $\mathbf{x}$, the four vertices of a box:

$$\mathbf{x} + \tfrac{1}{2}(\pm\Delta_1,\ \pm\Delta_2).$$

Here $\mathbf{x} = (0, 0)$, $\Delta = (2, 2)$, on Himmelblau.



From a shape-changing simplex to a box that does not change

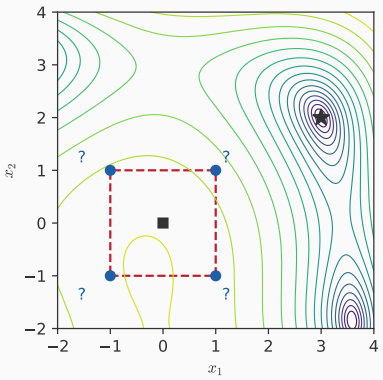
Nelder–Mead adapts the shape of the simplex using the values of f. What if we always keep the same geometric pattern?

In 2D, around the current point $\mathbf{x}$, the four vertices of a box:

$$\mathbf{x} + \tfrac{1}{2}(\pm\Delta_1,\ \pm\Delta_2).$$

Here $\mathbf{x} = (0, 0)$, $\Delta = (2, 2)$, on Himmelblau.

Which one would we choose?



From a shape-changing simplex to a box that does not change

Nelder–Mead adapts the shape of the simplex using the values of f. What if we always keep the same geometric pattern?

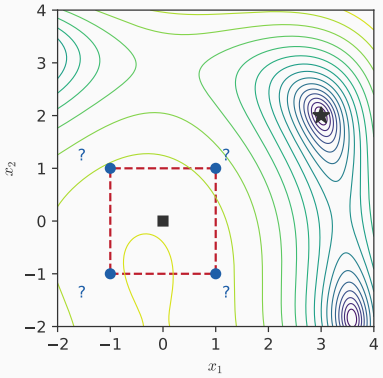
In 2D, around the current point $\mathbf{x}$, the four vertices of a box:

$$\mathbf{x} + \tfrac{1}{2}(\pm\Delta_1,\ \pm\Delta_2).$$

Here $\mathbf{x} = (0, 0)$, $\Delta = (2, 2)$, on Himmelblau.

Which one would we choose?

$\rightarrow$ the one with the smallest f — if it improves on the centre.



From a shape-changing simplex to a box that does not change

Nelder–Mead adapts the shape of the simplex using the values of f. What if we always keep the same geometric pattern?

In 2D, around the current point $\mathbf{x}$, the four vertices of a box:

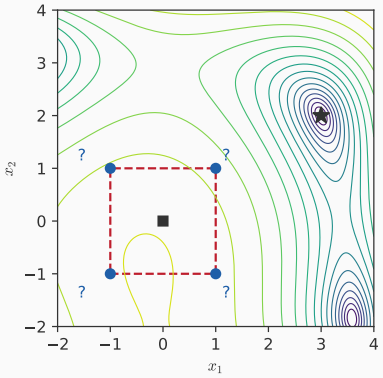
$$\mathbf{x} + \tfrac{1}{2}(\pm\Delta_1,\ \pm\Delta_2).$$

Here $\mathbf{x} = (0, 0)$, $\Delta = (2, 2)$, on Himmelblau.

Which one would we choose?

$\rightarrow$ the one with the smallest f — if it improves on the centre.

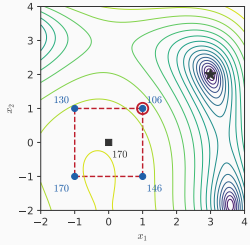
random search (3.2.1)	points without structure
grid (3.2.2)	fixed structure on the whole domain
Nelder–Mead (3.2.3)	adaptive geometry, which moves
Box (3.2.4)	fixed pattern, which moves



One iteration by hand

$f(\mathbf{x}) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \quad \mathbf{x} = (0, 0),$
$f(\mathbf{x}) = 170, \quad \Delta = (2, 2).$

vertex	f
$(-1, -1)$	170
$(1, -1)$	146
$(-1, 1)$	130
$(1, 1)$	106

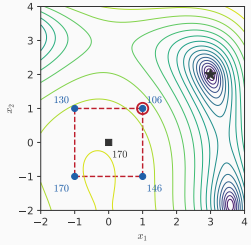


One iteration by hand

$f(\mathbf{x}) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \quad \mathbf{x} = (0, 0),$
$f(\mathbf{x}) = 170, \quad \Delta = (2, 2).$

vertex	f
$(-1, -1)$	170
$(1, -1)$	146
$(-1, 1)$	130
$(1, 1)$	106

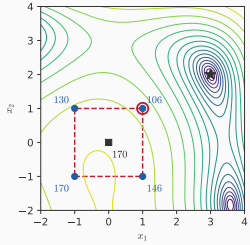
The best, $f(1, 1) = 106 < 170$: set $\mathbf{x} = (1, 1)$, and Δ stays the same.



One iteration by hand

$f(\mathbf{x}) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \quad \mathbf{x} = (0, 0),$
$f(\mathbf{x}) = 170, \quad \Delta = (2, 2).$

vertex	f
$(-1, -1)$	170
$(1, -1)$	146
$(-1, 1)$	130
$(1, 1)$	106



The best, $f(1, 1) = 106 < 170$: set $\mathbf{x} = (1, 1)$, and Δ stays the same.

And if none of the four were better? set $\Delta = \Delta/2$

One iteration by hand

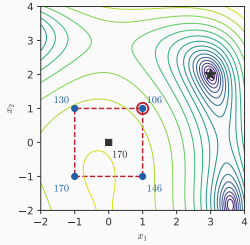
$f(\mathbf{x}) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2, \quad \mathbf{x} = (0, 0),$
$f(\mathbf{x}) = 170, \quad \Delta = (2, 2).$

vertex	f
$(-1, -1)$	170
$(1, -1)$	146
$(-1, 1)$	130
$(1, 1)$	106

The best, $f(1, 1) = 106 < 170$: set $\mathbf{x} = (1, 1)$, and Δ stays the same.

And if none of the four were better? set $\Delta = \Delta/2$

We have just built the whole algorithm.



Origin

G. E. P. Box (1957), *Evolutionary Operation*: tuning industrial processes *in production* (Box worked at ICI), with small variations around the current conditions; each evaluation was a batch. The original EVOP is experimental and statistical; here we use a simplified deterministic version (not to be confused with the *Complex* method of M. J. Box, 1965 — another author, another algorithm).

The algorithm, and its cost

Box: starting point $\mathbf{x}_0$, sides Δ_i, tolerance ε_x

Step 1 Set $\mathbf{x} = \mathbf{x}_0$; compute $f(\mathbf{x})$.

Step 2 If $\|\boldsymbol{\Delta}\| < \varepsilon_x$, **stop**: the solution is $\mathbf{x}$.

Step 3 Generate the 2^n vertices $\mathbf{x} + \frac{1}{2}(\pm\Delta_1, \ldots, \pm\Delta_n)$.

Step 4 Evaluate f at the vertices; let $\mathbf{x}'$ be the best.

Step 5 If $f(\mathbf{x}') < f(\mathbf{x})$, set $\mathbf{x} = \mathbf{x}'$ (move). Otherwise, set $\boldsymbol{\Delta} = \boldsymbol{\Delta}/2$ (shrink). Go back to step 2.

- The Δ_i may differ per coordinate; stop when the box becomes smaller than ε_x.
- An implementation with a cache can reuse points already evaluated; this deck's counts always evaluate all 2^n vertices.

$$n_f = 2^n \text{ per iteration}$$

n	2	3	10	20
vertices	4	8	1 024	$\approx 10^6$

The algorithm, and its cost

Box: starting point $\mathbf{x}_0$, sides Δ_i, tolerance ε_x

Step 1 Set $\mathbf{x} = \mathbf{x}_0$; compute $f(\mathbf{x})$.

Step 2 If $\|\boldsymbol{\Delta}\| < \varepsilon_x$, **stop**: the solution is $\mathbf{x}$.

Step 3 Generate the 2^n vertices $\mathbf{x} + \frac{1}{2}(\pm\Delta_1, \ldots, \pm\Delta_n)$.

Step 4 Evaluate f at the vertices; let $\mathbf{x}'$ be the best.

Step 5 If $f(\mathbf{x}') < f(\mathbf{x})$, set $\mathbf{x} = \mathbf{x}'$ (move). Otherwise, set $\boldsymbol{\Delta} = \boldsymbol{\Delta}/2$ (shrink). Go back to step 2.

- The Δ_i may differ per coordinate; stop when the box becomes smaller than ε_x.
- An implementation with a cache can reuse points already evaluated; this deck's counts always evaluate all 2^n vertices.

$$n_f = 2^n \text{ per iteration}$$

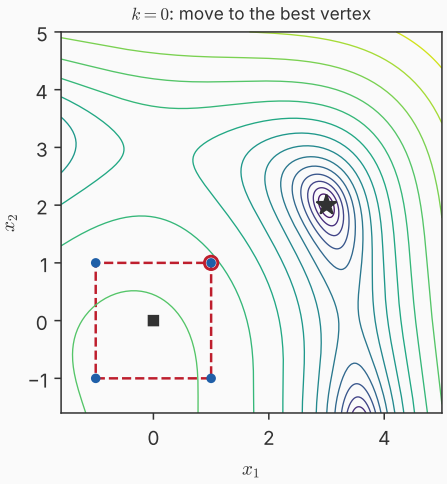
n	2	3	10	20
vertices	4	8	1 024	$\approx 10^6$

The curse of dimensionality again — for a different reason:

- random search and grid (3.2.1–3.2.2): the volume to cover;
- Box: the number of vertices to test.

Same consequence: dimension is very costly.

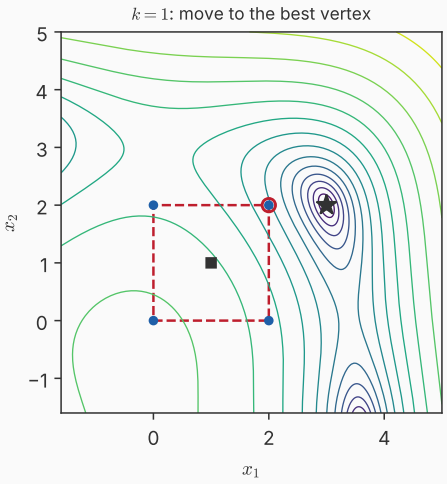
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000

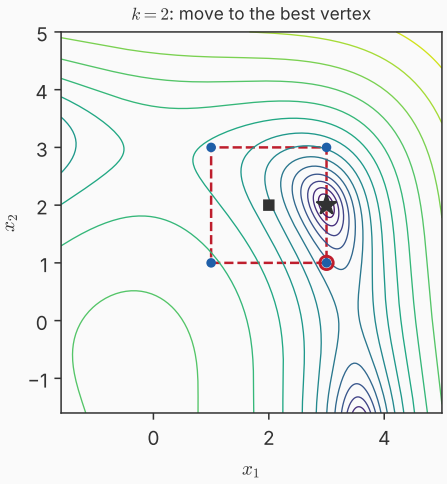
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000

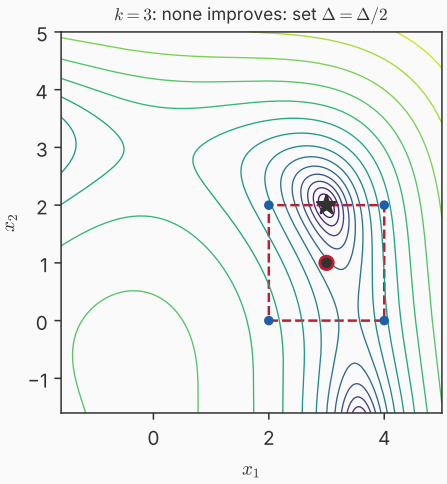
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000
2	(2.000, 2.000)	2.000	26.000	move	10.000

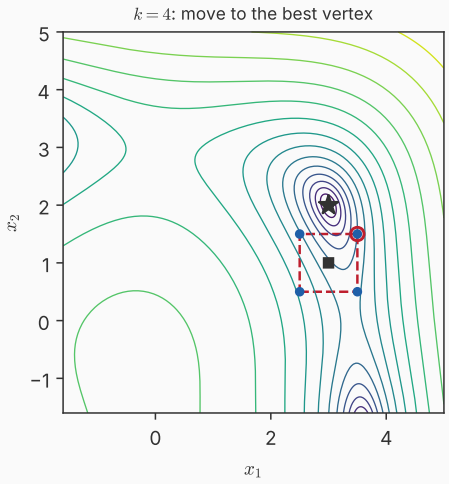
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000
2	(2.000, 2.000)	2.000	26.000	move	10.000
3	(3.000, 1.000)	2.000	10.000	$\Delta = \Delta/2$	10.000

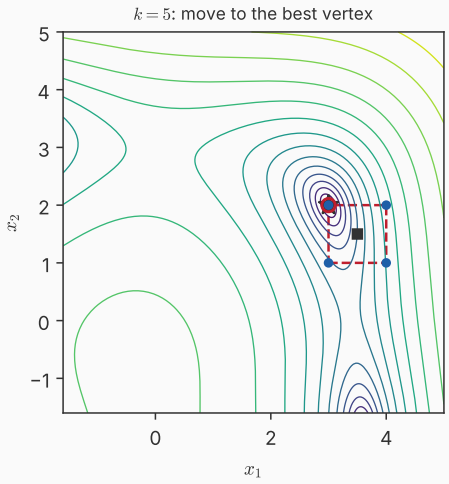
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000
2	(2.000, 2.000)	2.000	26.000	move	10.000
3	(3.000, 1.000)	2.000	10.000	$\Delta = \Delta/2$	10.000
4	(3.000, 1.000)	1.000	10.000	move	9.125

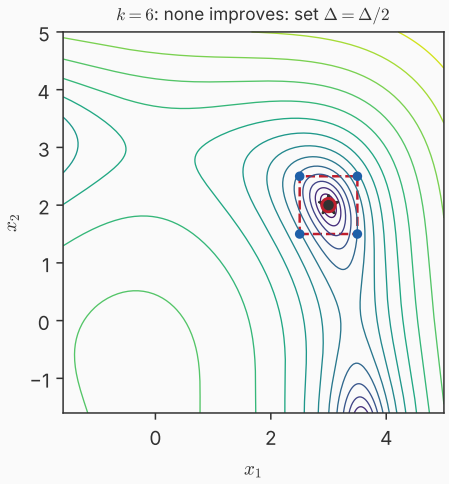
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000
2	(2.000, 2.000)	2.000	26.000	move	10.000
3	(3.000, 1.000)	2.000	10.000	$\Delta = \Delta/2$	10.000
4	(3.000, 1.000)	1.000	10.000	move	9.125
5	(3.500, 1.500)	1.000	9.125	move	0.000

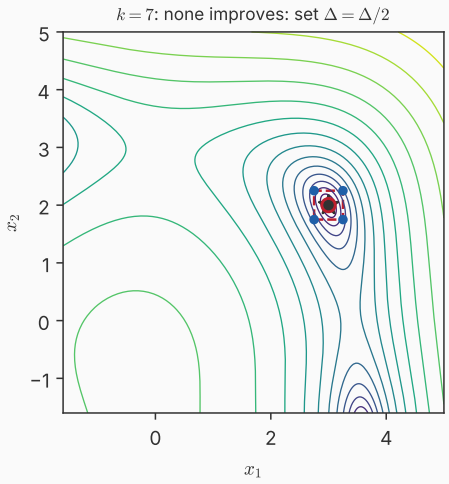
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000
2	(2.000, 2.000)	2.000	26.000	move	10.000
3	(3.000, 1.000)	2.000	10.000	$\Delta = \Delta/2$	10.000
4	(3.000, 1.000)	1.000	10.000	move	9.125
5	(3.500, 1.500)	1.000	9.125	move	0.000
6	(3.000, 2.000)	1.000	0.000	$\Delta = \Delta/2$	0.000

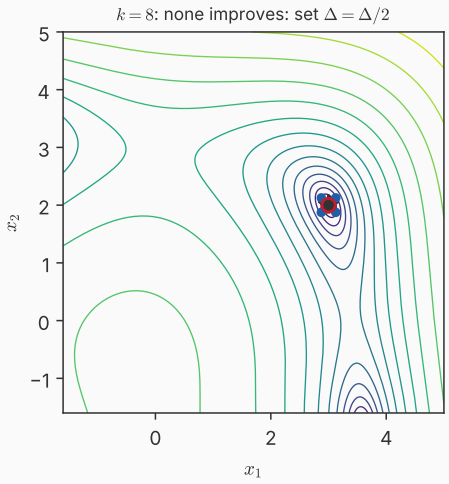
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000
2	(2.000, 2.000)	2.000	26.000	move	10.000
3	(3.000, 1.000)	2.000	10.000	$\Delta = \Delta/2$	10.000
4	(3.000, 1.000)	1.000	10.000	move	9.125
5	(3.500, 1.500)	1.000	9.125	move	0.000
6	(3.000, 2.000)	1.000	0.000	$\Delta = \Delta/2$	0.000
7	(3.000, 2.000)	0.500	0.000	$\Delta = \Delta/2$	0.000

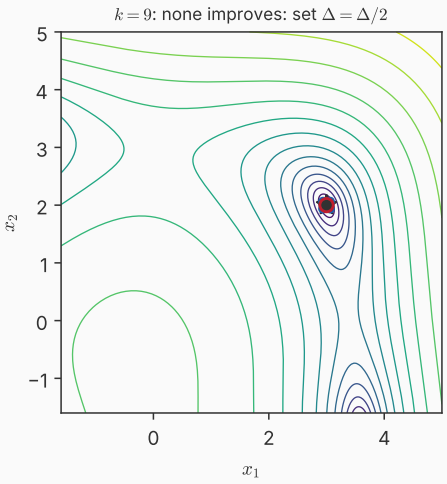
Himmelblau from $(0, 0)$: move or shrink



$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000
2	(2.000, 2.000)	2.000	26.000	move	10.000
3	(3.000, 1.000)	2.000	10.000	$\Delta = \Delta/2$	10.000
4	(3.000, 1.000)	1.000	10.000	move	9.125
5	(3.500, 1.500)	1.000	9.125	move	0.000
6	(3.000, 2.000)	1.000	0.000	$\Delta = \Delta/2$	0.000
7	(3.000, 2.000)	0.500	0.000	$\Delta = \Delta/2$	0.000
8	(3.000, 2.000)	0.250	0.000	$\Delta = \Delta/2$	0.000

Himmelblau from $(0, 0)$: move or shrink

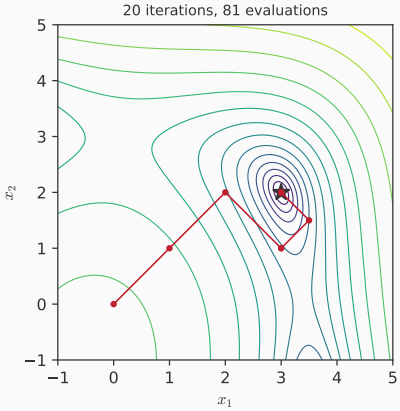


$\Delta = (2, 2)$, $\varepsilon_x = 10^{-4}$. In each row, the centre $\mathbf{x}_k$ and the decision taken from it.

k	$\mathbf{x}_k$	Δ	$f(\mathbf{x}_k)$	decision	new f
0	(0.000, 0.000)	2.000	170.000	move	106.000
1	(1.000, 1.000)	2.000	106.000	move	26.000
2	(2.000, 2.000)	2.000	26.000	move	10.000
3	(3.000, 1.000)	2.000	10.000	$\Delta = \Delta/2$	10.000
4	(3.000, 1.000)	1.000	10.000	move	9.125
5	(3.500, 1.500)	1.000	9.125	move	0.000
6	(3.000, 2.000)	1.000	0.000	$\Delta = \Delta/2$	0.000
7	(3.000, 2.000)	0.500	0.000	$\Delta = \Delta/2$	0.000
8	(3.000, 2.000)	0.250	0.000	$\Delta = \Delta/2$	0.000
9	(3.000, 2.000)	0.125	0.000	$\Delta = \Delta/2$	0.000

The box moves with fixed size,
$(0, 0) \to (1, 1) \to (2, 2) \to (3, 1)$; at $(3, 1)$ no vertex improves and it shrinks (Δ: $2 \to 1$); then
$(3, 1) \to (3.5, 1.5) \to (3, 2)$ and it only shrinks.

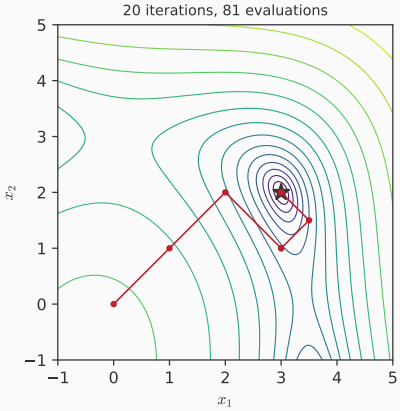
The complete path: what the box is doing



20 iterations, $n_f = 4 \times 20 + 1 = 81$ until $\|\boldsymbol{\Delta}\| < 10^{-4}$.

- While there is improvement, the centre jumps from vertex to vertex: all coordinates change by $\pm\Delta_i/2$, so the step has fixed length $\frac{1}{2}\,\|\boldsymbol{\Delta}\|$. Only when it stagnates does it decrease.
- It reaches $(3, 2)$ *exactly* because the minimum lies on the grid of steps $\Delta/2$.

The complete path: what the box is doing

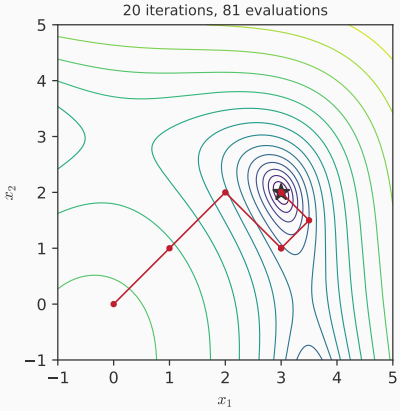


20 iterations, $n_f = 4 \times 20 + 1 = 81$ until $\|\mathbf{\Delta}\| < 10^{-4}$.

- While there is improvement, the centre jumps from vertex to vertex: all coordinates change by $\pm\Delta_i/2$, so the step has fixed length $\frac{1}{2}\,\|\mathbf{\Delta}\|$. Only when it stagnates does it decrease.
- It reaches $(3, 2)$ *exactly* because the minimum lies on the grid of steps $\Delta/2$.

If Δ is very small, do we know we are close to $\mathbf{x}^*$?

The complete path: what the box is doing



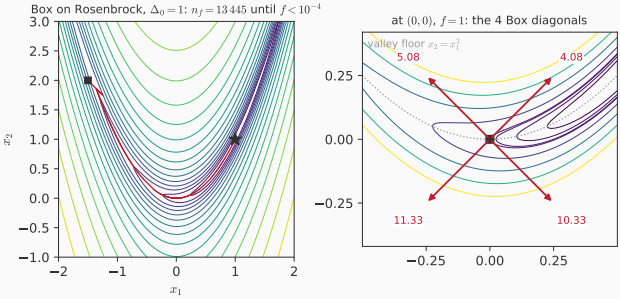
20 iterations, $n_f = 4 \times 20 + 1 = 81$ until $\|\mathbf{\Delta}\| < 10^{-4}$.

- While there is improvement, the centre jumps from vertex to vertex: all coordinates change by $\pm\Delta_i/2$, so the step has fixed length $\frac{1}{2}\,\|\mathbf{\Delta}\|$. Only when it stagnates does it decrease.
- It reaches $(3, 2)$ *exactly* because the minimum lies on the grid of steps $\Delta/2$.

If Δ is very small, do we know we are close to $\mathbf{x}^*$?

No. We only know that, in that pattern and at that scale, no tested vertex improved. A small box does not certify stationarity at finite Δ, nor a local minimum, nor $\|\mathbf{x} - \mathbf{x}^*\| = O(\varepsilon_x)$.

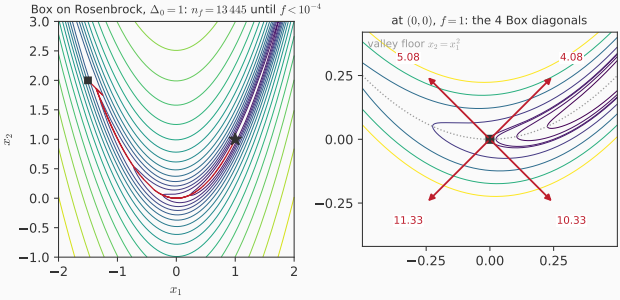
Why does it fail in curved valleys?



What is wrong with changing all coordinates at once?
On Rosenbrock, from $(-1.5, 2)$ with $\Delta_0 = 1$, the box has to shrink from 1 to $\approx 6 \cdot 10^{-5}$ to make progress in the valley, and then it advances in tiny steps: $n_f = 13\,445$ until $f < 10^{-4}$ (with $\varepsilon_x = 10^{-6}$).

At $(0, 0)$, $\Delta = (0.5, 0.5)$ (vertices ± 0.25): no diagonal $(\pm 1, \pm 1)$ improves $f = 1$ (4.08 to 11.33).

Why does it fail in curved valleys?

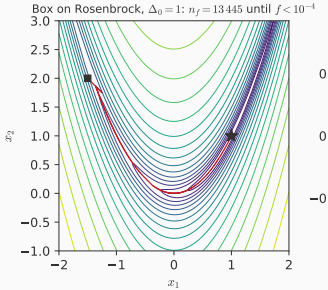


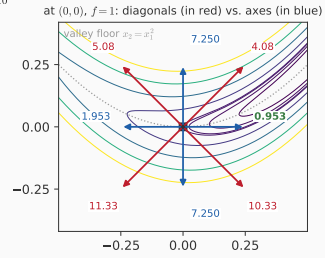
What is wrong with changing all coordinates at once? On Rosenbrock, from $(-1.5, 2)$ with $\Delta_0 = 1$, the box has to shrink from 1 to $\approx 6 \cdot 10^{-5}$ to make progress in the valley, and then it advances in tiny steps: $n_f = 13\,445$ until $f < 10^{-4}$ (with $\varepsilon_x = 10^{-6}$).

At $(0, 0)$, $\Delta = (0.5, 0.5)$ (vertices ± 0.25): no diagonal $(\pm 1, \pm 1)$ improves $f = 1$ (4.08 to 11.33).

Box concluded that there is no improvement. Is there really no descent direction?

Why does it fail in curved valleys?





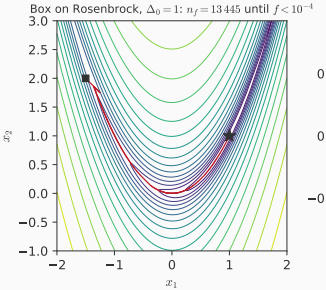
What is wrong with changing all coordinates at once?
On Rosenbrock, from $(-1.5, 2)$ with $\Delta_0 = 1$, the box has to shrink from 1 to $\approx 6 \cdot 10^{-5}$ to make progress in the valley, and then it advances in tiny steps: $n_f = 13\,445$ until $f < 10^{-4}$ (with $\varepsilon_x = 10^{-6}$).

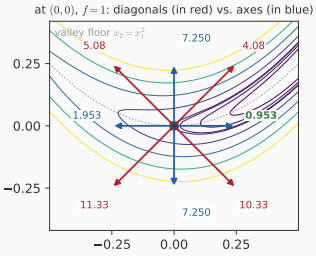
At $(0, 0)$, $\Delta = (0.5, 0.5)$ (vertices ± 0.25): no diagonal $(\pm 1, \pm 1)$ improves $f = 1$ (4.08 to 11.33).

Box concluded that there is no improvement. Is there really no descent direction?

There is: the axis $+\mathbf{e}_1$ improves, $f(0.25, 0) = 0.953$. The valley is horizontal there; the diagonals leave it.

Why does it fail in curved valleys?





What is wrong with changing all coordinates at once?

On Rosenbrock, from $(-1.5, 2)$ with $\Delta_0 = 1$, the box has to shrink from 1 to $\approx 6 \cdot 10^{-5}$ to make progress in the valley, and then it advances in tiny steps: $n_f = 13\,445$ until $f < 10^{-4}$ (with $\varepsilon_x = 10^{-6}$).

At $(0, 0)$, $\Delta = (0.5, 0.5)$ (vertices ± 0.25): no diagonal $(\pm 1, \pm 1)$ improves $f = 1$ (4.08 to 11.33).

Box concluded that there is no improvement. Is there really no descent direction?

There is: the axis $+\mathbf{e}_1$ improves, $f(0.25, 0) = 0.953$. The valley is horizontal there; the diagonals leave it.

diagonals $(\pm 1, \ldots, \pm 1)$ vs. axes $\pm \mathbf{e}_i$

The price of Box; on the computer

Advantages

order 0; deterministic; very simple; scale Δ_i controllable per variable; the evaluations of the 2^n vertices can be done in parallel.

Limitations

- 2^n evaluations per iteration;
- only the 2^n diagonals $D = \{(\pm 1, \ldots, \pm 1)\}$: depends on the coordinate orientation;
- inefficient in narrow, curved valleys; local;
- small box $\not\Rightarrow$ stationarity (at finite Δ);
- noise can change the choice of the best vertex.

On Rosenbrock from $(-1.5, 2)$, until $f < 10^{-4}$: Box ($\Delta_0 = 1$) $n_f = 13\,445$; Nelder–Mead $n_f = 244$ (same problem, same starting point, same criterion; with these parameters).

The core, in MATLAB:

```
S = dec2bin(0:2^n-1) - '0';
S = 2*S - 1;                % rows of +-1
V  = x + 0.5*S.*Delta;      % vertices
Fv = arrayfun(@(i) f(V(i,:)), 1:2^n);
[fmin, i] = min(Fv);
if fmin < fx
    x = V(i,:);  fx = fmin;   % move
else
    Delta = Delta/2;          % shrink
end
```

In 2D, `S` has the 4 rows $(-1,-1)$, $(-1,1)$, $(1,-1)$, $(1,1)$.

The full function (`BoxEvo`, with `info.nfev`) is in the course companion code.

KEY TAKEAWAYS — 3.2.4

1. Box uses a box centred at the current point and tests all 2^n diagonal combinations.
2. Improvement $\Rightarrow$ move to the best vertex; failure $\Rightarrow$ set $\Delta = \Delta/2$.
3. It uses only values of f, but the cost, 2^n per iteration, grows exponentially with the dimension: testing all diagonals is expensive and rigid.
4. A small box does not certify stationarity at finite Δ. For this simplified version of Box we do not present here a general stationarity guarantee.
5. NM has no general guarantee; HJ is the origin of pattern search, whose formalized versions (GPS) have, under suitable assumptions, $\liminf \|\nabla f(\mathbf{x}_k)\| = 0$ (3.2.5).

Next (3.2.5): do we really need the 2^n diagonals? What if we explore one coordinate at a time, with only $2n$ directions, and then exploit the direction of progress? — the Hooke–Jeeves method.

References and resources

References Full list: *Course bibliography*, at the end of the course PDF.

- Box, G. E. P., *Evolutionary operation: a method for increasing industrial productivity*, Journal of the Royal Statistical Society, Series C (Applied Statistics) 6(2):81–101, 1957.
- Deb, K., *Optimization for Engineering Design: Algorithms and Examples*, 2nd ed., PHI Learning, 2012 — Ch. 3.

Resources

- Companion code: `BoxEvo` (`info.nfev`).